ARC-AGI Milestone D

Scott Schmidl
sschmidl3@gatech.edu

1 HOW DOES YOUR AGENT CURRENTLY FUNCTION?

My milestone D agent keeps the same overall structure as Milestone C. It is still rule-based and treats the problems as constrained hypothesis testing over a finite bank of transformations. For each problem, the agent loads the full set of training pairs along with the test input. It then iterates over the rule bank specific to milestone D and keeps only the rules that reproduce the outputs for every training example exactly.

The key difference in Milestone D is the rules the agent now considers. The D rule bank includes those that are good at relational structure, path tracing, logical overlap, and enclosed region reasoning. For example, `xor6` compares the top and bottom halves of a six-row grid using a logical exclusive OR operation, `panelmirror` splits a grid into panels using separator rows or columns and reconstructs the panel by symmetry, `trace2` connects marked cells with a path, and `holey1` detects connected components and fills enclosed holes inside them.

The agent also relies on the same shared utilities as before. These utilities identify connected components, return coordinates, compute bounding boxes, and label components. Basically, the utilities give the rules a more abstract description of the grid, so rules don't have to work from raw pixels.

The agent maintains the rule bank and a bank of builders. The builders are used to generate additional rules from recurring structures, including color filtering and markers. The final predictions are made such that any candidate that fails on even one training pair is removed, while the remaining are applied to the test grid, duplicates get removed, and at most three predictions are returned.

2 HOW WELL DOES YOUR AGENT CURRENTLY PERFORM?

For this milestone the agent solves correctly 16 out of 16 local problems and 9 out of 16 hidden cases. The seven hidden failures are on `60a26a3e`, `992798f6`, `c1990cce`, `bcb3040b`, `e9b4f6fc`, `d931c21c`, and `2546ccf6`. Overall, I think this is so far an encouraging result. The score on the local cases shows that the current rule bank is able to capture all Milestone D patterns at least once. At the same

time, the drop to 9 out of 16 on the hidden cases means that several of the rules are too specific, especially on tasks where the transformations don't generalize well from the local training pairs.

When compared with Milestone C, I can see similar results but clear areas where improvement needs to be made. Milestone D clearly contains more cases where passing local cases is not enough. In other words, Milestone D is stressing more generalization rather than the previously seen straight rule matching.

3 WHAT PROBLEMS DOES YOUR AGENT PERFORM WELL ON?

My agent performs best on Milestone D problems where the transformation can still be expressed as a single rule with a geometric or relational structure. Tasks include mirroring across a boundary and path tracing. In these cases, the training pairs are passed through one rule in the bank and that rule is able to generalize.

My agent also does well on problems where the concept stays stable from train to test even if the grid changes, for example on tasks where there is a missing panel or a pattern to be copied into a symmetric location. For these cases, once the agent has seen three consistent panels in the training pairs, the same behavior tends to work on a larger or shifted test grid.

It also performs well on problems where there is a single relationship between a small number of objects. One good example is where the output is based on how two shapes overlap or by how a path should connect two markers, regardless of where they are on the grid. When the training pairs make that relationship clear, the agent is able to find the pattern and reproduce it reliably on the hidden cases.

4 WHAT PROBLEMS (IF ANY) DOES IT STRUGGLE ON?

The agent struggles most on problems where the correct solution is more abstract than the patterns it sees in the training pairs, or where the transformation is a composition of multiple steps rather than a single rule. This can be seen in the seven hidden cases that fail.

For instance, in `992798f6` the training pairs show routing a path between blue and red markers, but the hidden case might use a different orientation or bend. My `trace2` rule currently only uses a specific path, so it fits the training pairs but doesn't generalize when the hidden case uses a different path.

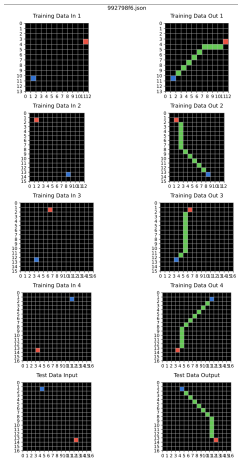


Figure 1—992798f6 - Draw Line from Blue to Diagonal Neighbor of Red

Similarly, in `d931c21c` the agent learns to wrap around a fixed pattern of colors based on specific sizes and aspect ratios, but the hidden cases might change the loop geometry in such a way that my current rule doesn't capture, potentially causing the shells to be misplaced or missing.

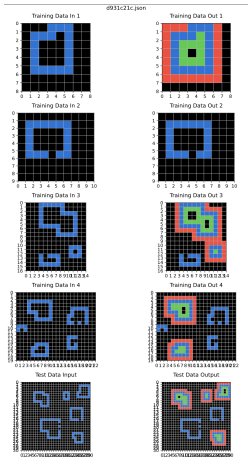


Figure 2—d931c21c - If Closed Fill One Layer In (Green) and Out (Red) Else Do Nothing

Another recurring issue is in problems like `2546ccf6`, where objects live inside a grid and the agent must copy specific cells into the missing quadrant. My rules reason about connected components and bounding boxes, but might not model grids-of-grids well enough, so they assume fixed colors or targets that happen to

work on training pairs but not hidden cases. More broadly speaking, the agent has trouble when the solution depends on relationships between multiple objects rather than a single transformation.

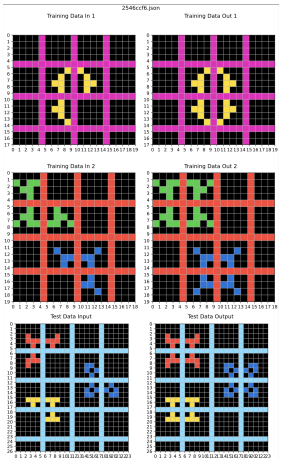


Figure 3—2546ccf6 - Fill Missing Quadrant

Another hidden case failure is `e9b4f6fc`, which requires the agent to crop down to the colored square and then swap the inner colored patterns according to which colors appear as adjacent pairs in the single markers. This is difficult for my current implementation because the task requires first identifying the relevant square region, then understanding that the smaller color pairs are role information, and finally remapping the inner pattern using those roles. My current rule can match the surface structure, but struggles with this kind of color-role swapping. As a result, the learned rule is too tied to exact layouts in the training data and doesn't generalize well to the hidden case.

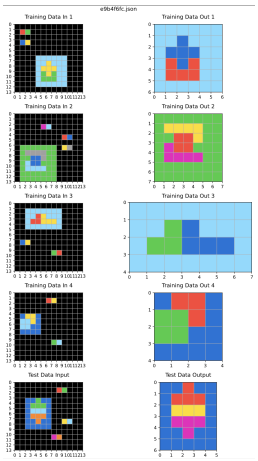


Figure 4—e9b4f6fc - Crop to Colored Square and Swap Adjacent Colors

5 HOW EFFICIENT IS YOUR AGENT?

Despite the bigger rule bank, the agent remains efficient. The overall implementation still processes each problem, iterates over a finite rule bank, applies each rule to every training pair, and discards any rule that fails to reproduce all training outputs. Since rules are functions over grids and most of them run in linear time, the time complexity is roughly O(R * C), where R is the number of rules and C is the total number of cells across all training grids.

Milestone D runs locally in about 0.0615 seconds and about 0.1196 seconds on the full suite of local and hidden tests. This means that each problem only takes a few milliseconds per problem on average. Due to this quick time, I wasn't able to notice any problem that causes a noticeable slowdown. Right now the rule bank is manageable and I don't expect that to change even if I'm able to solve the other seven problems. There could be some risk if the remaining problems require several compositions. In which case the rule bank could grow very large and increases the time complexity to a less desirable Big-O, but right now the search space is small enough that the performance is effectively instantaneous.

6 HOW DO YOU PLAN TO IMPROVE YOUR AGENT'S PERFORMANCE?

Before the final project submission, I plan to focus on two main improvements. First, I want to make the existing D rules more object-centric and color agnostic so they're better at generalizing to new layouts and colors rather than overfitting.

For instance, `trace2` could treat a path as a shortest path problem between two markers, with routing preferences learned from examples. Similarly, the wrapping behavior in problems like `d931c21c` could be written to reason about one layer inside and one outside each detected loop, independent of size or aspect ratio.

Secondly, I want to introduce some rule composition. Right now, each rule is a single transformation which makes reasoning simple but limits the kinds of patterns the agent can find. Several D problems look like they decompose naturally, i.e. detect or normalize an object, then apply another transform, then embed the result into a large structure. Problems such as `2546ccf6` could be good candidates for a pipeline that will detect grid cells, then index objects by cell, and then copy to a new cell. I'd like to experiment with some compositions guided by features of the training pairs, so the search space grows but stays manageable.

One final thing I could do is add debugging, such as logging rules that match all training pairs but one, or recording where a predicted grid is different than the ground truth. This should be able to help me see whether a rule is close but just missing a small piece, or whether I need to use a completely different strategy. Having logging information should help guide which rules need refinement versus needing new rules or compositions.

7 WHAT FEEDBACK WOULD YOU HOPE TO GET FROM CLASSMATES?

From classmates, I would like feedback on two areas. The first is how to introduce rule compositions without letting the search space explode. I am interested in strategies that let me chain a few current rules while still using the training pairs as filters, rather than brute-forcing all possible compositions. Concrete examples of how others are structuring their compositions would be helpful.

The second area is representation, i.e. with patterns like `2546ccf6` I don't have the right middle representation. I would like suggestions of compact representations that make some of the D problems easier to express, as well as experiences with using meta-reasoning to help decide which group of rules to try first on a new problem.